

QCC Labs Reverse Engineering iOS, Android, and more

Author: @MicahKBrown (X)

Goal: Attendee will gain real world experience in reverse engineering iOS and Android apps

Pre Requirements:

- Personal laptop
- Install VirtualBox (or other preferred hypervisor)
- Load a working REMnux VM into your hypervisor

Reverse engineering is the process of deconstructing an object, system, or software to understand its design, components, and functionality by working backward from the finished product, often to recreate, improve, or analyze it without original blueprints. It involves detailed analysis, disassembly, and documentation to extract design information for various purposes, like developing competing products, fixing old tech, or identifying security flaws.

Hey Team, @MicahKBrown here! Within ITSec, we have so many disciplines and no one can have excellence in all disciplines. Most people will have a small number of disciplines where they have excellence. Then they will have competence in a few more disciplines and then a handful of disciplines where we have little to no excellence. For me, my weakest discipline in ITSec is Application Security. Why do I tell you this, it is to set expectations. This is an entry level lab designed to help you explore the very start of Reverse Engineering tools, tactics, and techniques.

REMnux

“

REMnux: A Linux Toolkit for Malware Analysis

- REMnux® is a Linux toolkit for reverse-engineering and analyzing malicious software. REMnux provides a curated collection of free tools created by the community. Analysts can use it to investigate malware without having to find, install, and configure the tools.

“

https://remnux.org/

REMnux has several options to use:

- Download the Virtual Appliance (Generic OVA / VirtualBox OVA)

- Run as a container
- Install from scratch

All instructions are fully documented on their friendly webpage!

Hi team, post editing Micah here! We are a little more than two months away from QCC 0x4 and I am excited. This year I have taken significant strides to increase the quality of labs by increasing network speed of lab files, making lab files easier, dedicated trainer machines fully set up, and even loaner laptop for those that want to play. So as I was building out a new version of this lab environment I noticed REMnux had released a new version of their OS (v8). Unfortunately, they took away some critical resources for this lab, most specifically removing the console and removing firefox. Thanks to being a digital packrat, I was able to find v7 of REMnux which was what this lab is built on.

In the real world my advise to you would be to leverage v8. But for simplicity of getting started, I recommend using the lab's ethernet switch to download v7 as it will make this lab much simpler.

At the time of writing the VirtualBox Appliance (OVA requires 4096MB of RAM and 60 GB virtual hard drive.

iOS Apps

Making an iOS app involves using Apple's Xcode IDE on a Mac to code in Swift (or Objective-C), design the user interface (UI) with tools like SwiftUI, connect UI elements to logic, test using built-in simulators, and then package and submit it to the App Store via the Apple Developer Program. The process flows from idea to design, development (coding/UI building), testing on simulators, and finally distribution.

Key Tools & Languages

- Xcode: The official Integrated Development Environment (IDE) from Apple, containing everything needed: editor, debugger, simulators, and tools.
- Swift: Apple's modern, safe, and fast programming language for iOS development.
- SwiftUI: Apple's declarative framework for building user interfaces across all Apple platforms.

Generally iOS apps use an .ipa extension.

Unless jailbroken an apple (iPone, iPad, iWatch, apple TV) can only run apps 'purcased' for the Apple App Store.

Android Apps

Making an Android app involves defining an idea, setting up Android Studio, coding with Kotlin (recommended) or Java, designing the UI, implementing features, testing thoroughly, and finally publishing to the Google Play Store. The process uses the Android SDK to compile code and resources into an installable APK or App Bundle (AAB) for distribution, leveraging components like Activities, Services, and Content Providers for functionality
.

Generally Android apps use an .apk entension or an App Bundel (AAB) for distribution.

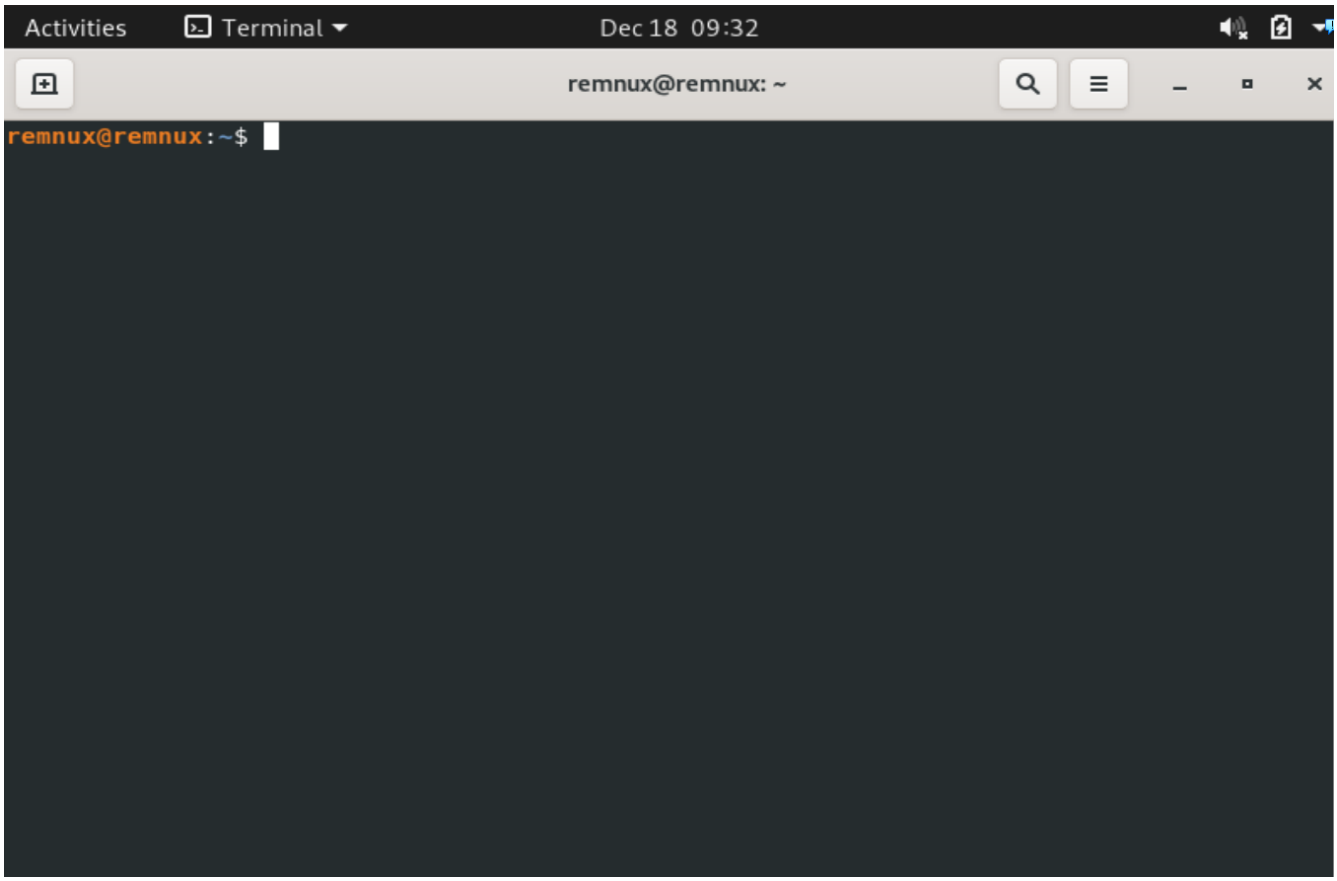
Most Android devices are locked to either the manufacture app store of the Google Play store. However, a user is generally allowed to turn that restriction off through settings and install apps from anywhere without needing to jailbreak the system.

Importing REMnux into VirtualBox

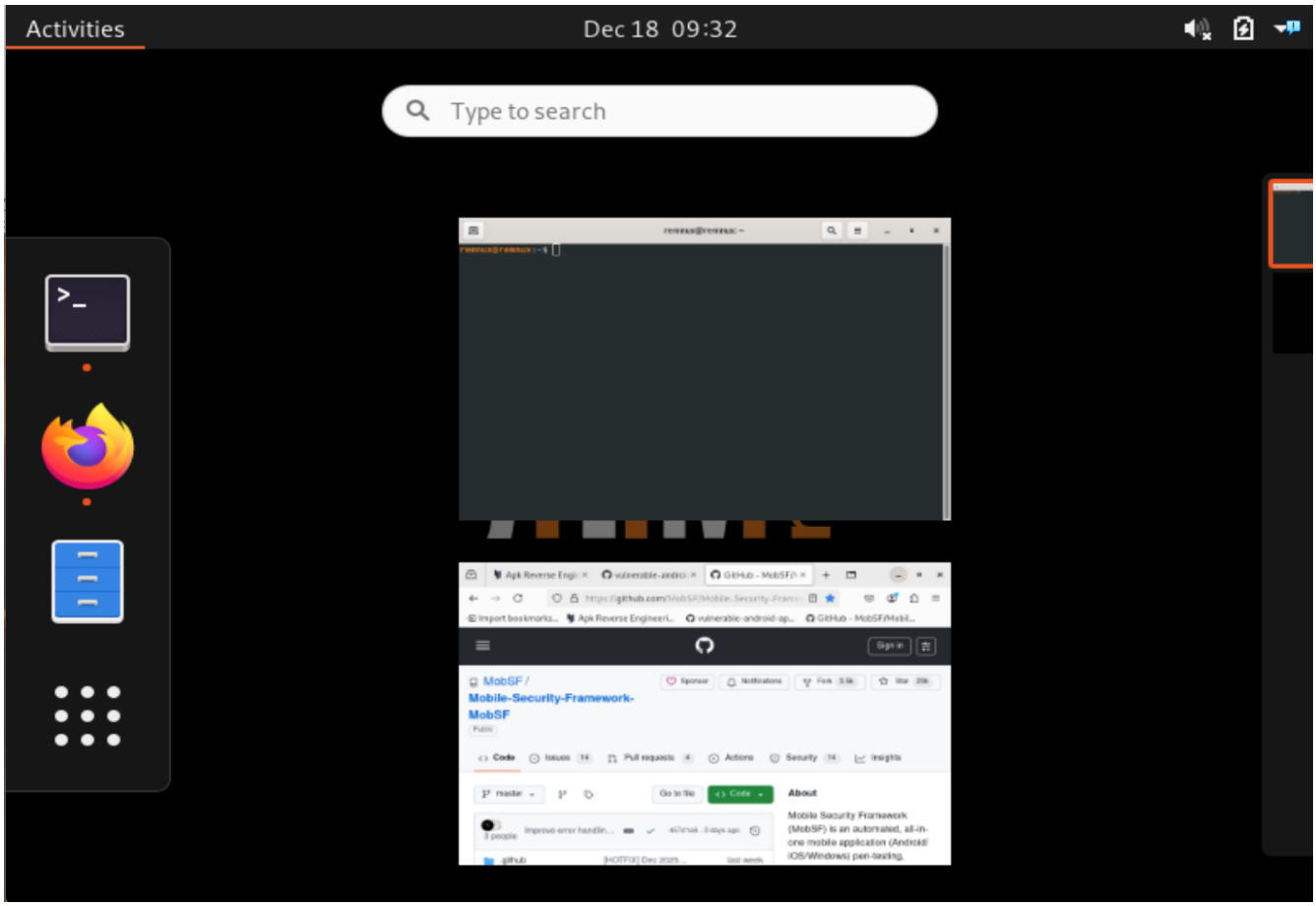
Make sure you are on the QCC Lab wifi.

Open Virtual Box.

- Click on File -> Import Appliance (or 'Control + I')
- On the pop up:
    - Under 'Source' -> 'File' line click the folder with the green up arrow. Then select your VirtualBox Appliance (OVA) file and click OK.
    - Expand 'Settings' -> 'Machine Base Folder' navigate to the location you want your VM to be built under.
- Click 'Finish'
- It should take about 5 to 10 minutes to import.
- Now Virtual Box click on the REMnux VM and then click settings button (looks like a gear). Click on networking and change 'Attached to' to 'Bridged Adapter'
- Once the VM is finished importing click on your REMnux VM and then click 'Start'
- The VM does not have a username or password and will load directly into a terminal session, this is normal.

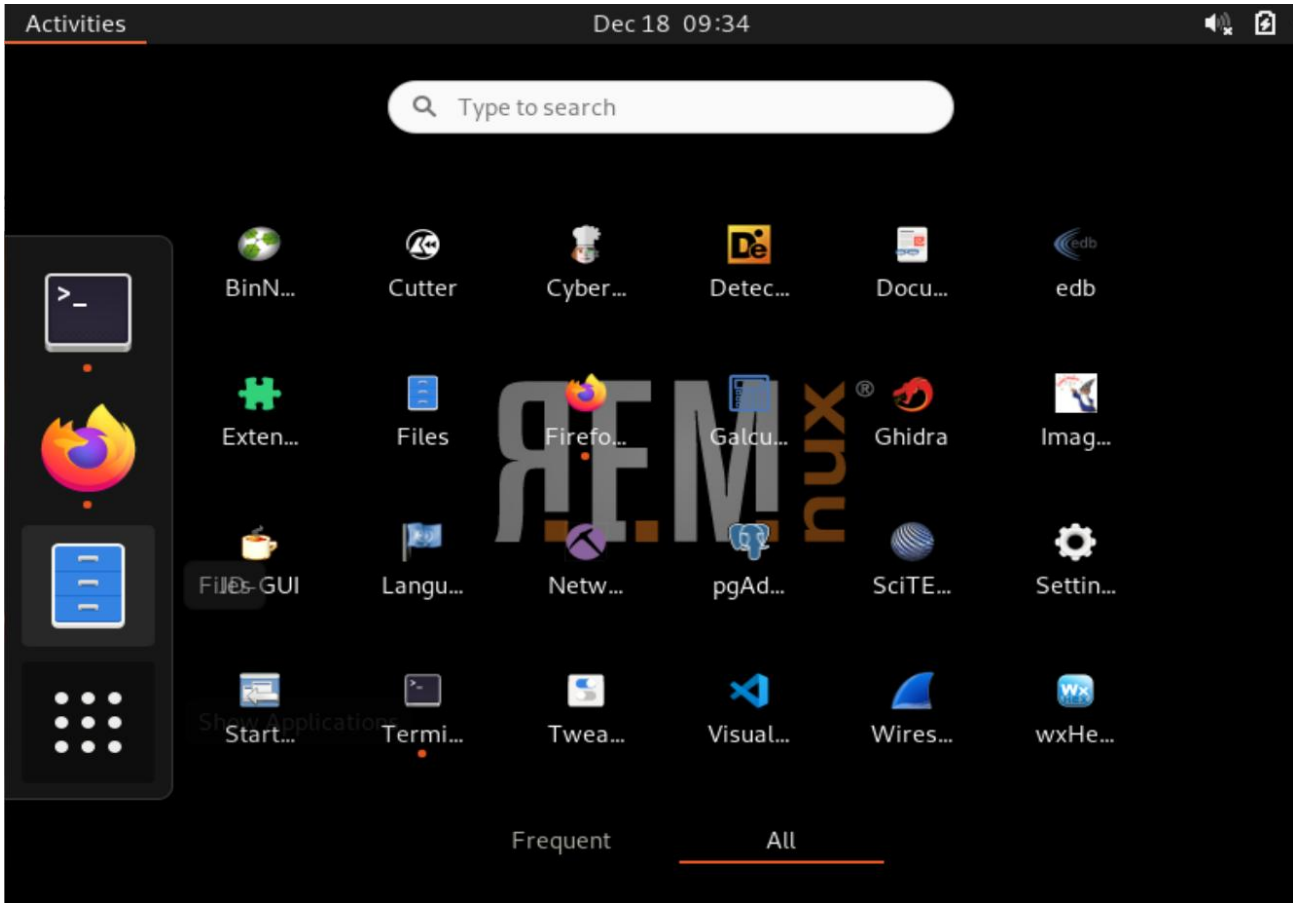


- From here you can click on ‘Activities’ to get to more options on the left: (description from top down)



  - Terminal Icon: launches new terminal session
  - Firefox: launches Firefox

- Nine dots in a square: Allows you to launch other pre-installed apps. (Will call this the ‘Launcher’ for the rest of the lab.



As you can see REMnux has several tools built in for free-fifty!

Now lets get the lab files copied to the desktop.

1. Make sure your host is connected to the QCC Wifi your REMnux v7 VM networking setting is set to bridged adaptor. (to change network adapters you must power off the VM, change the network type, and then power on the VM)
2. Open up firefox and navigate to https://192.168.8.250:1337 to connect to the QCC LocalSend file share
3. You get a warning because we are using a self signed certificate and are not paying one of the Cas to vouch for us. Using self-signed certificates does not necessarily decrease the strength of encryption. Click ‘Advanced’ and ‘Accept Risk and Continue’.
4. Locate the ‘QCCLabs/1_2026QCCLabFiles.zip’ and click on it. (if it is taking too long you can power down your VM connect the lab switch, power you REMnux VM up and download it. We just ask after you download your files, please disconnect and find another seat so another person can use the lab infrastructor)
5. Once the fie downloads, in the files app click on downloads. Then right click QCCLabs/1_2026QCCLabFiles.zip and select ‘Extract Here’ to unzip.
6. This will create a couple nested folders that we will deal with. Open folders until you find the QCC.Labs.Reverse.Engineering folder. Here you will find the lab guide and all files for this folder!

Challenge 1 & 2: Mobile-Security-Framework-MobSF

> “
> Mobile Security Framework (MobSF) is a security research platform for mobile applications in Android, iOS and Windows Mobile. MobSF can be used for a variety of use cases such as mobile application security, penetration testing, malware analysis, and privacy analysis. The Static Analyzer supports popular mobile app binaries like APK, IPA, APPX and source code. Meanwhile, the Dynamic Analyzer supports both Android and iOS applications and offers a platform for interactive instrumented testing, runtime data and network traffic analysis. MobSF seamlessly integrates with your DevSecOps or CI/CD pipeline, facilitated by REST APIs and CLI tools, enhancing your security workflow with ease.
> Made with ❤️ in India
> “

https://github.com/MobSF/Mobile-Security-Framework-MobSF

This is a great tool for us to start off with as it is literally drag and drag and drop using the web tool! Please note as this is a SaaS tool, any of the apps that you upload to this site are literally in the hands of the MobSF team. Even if you do a local install, you should consider that the results of the scan will be posted by the https://mobsf.live/ site for the entire internet. This might be to big of a security risk for business critical applications

https://mobsf.live/

If MobSF has already seen this exact app based on a hash / fingerprint, the results will be immediate. If this is a new file

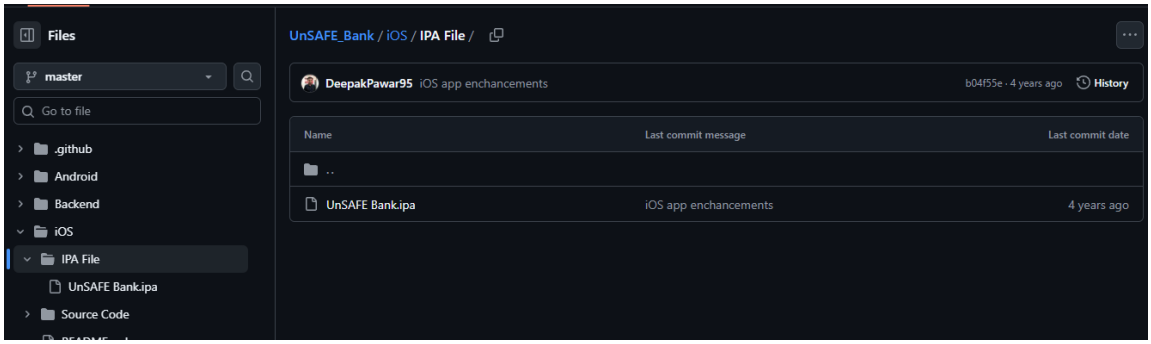
For this lab we are going to upload an iOS and an Android app.

It is important to remember if an app is encrypted, most Reverse Engineering tools will give us very limited results. Both of the file we will use in this step are unencrypted.

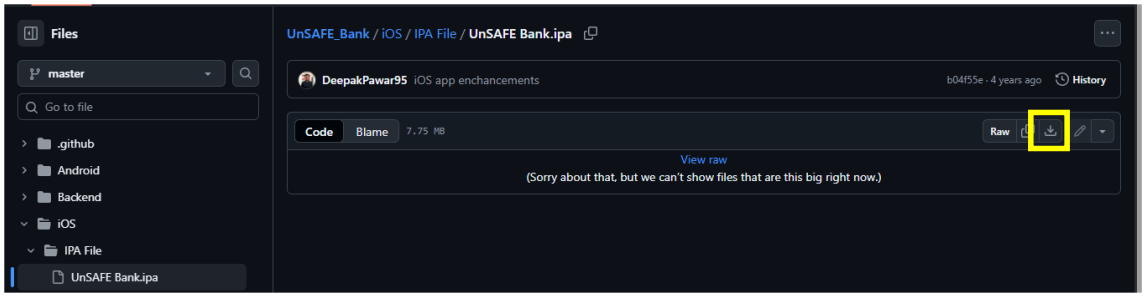
A special thanks to the team at https://github.com/topics/vulnerable-android-apps for providing us these two test files for this section!

From either your hypervisor host or REMnux VM:

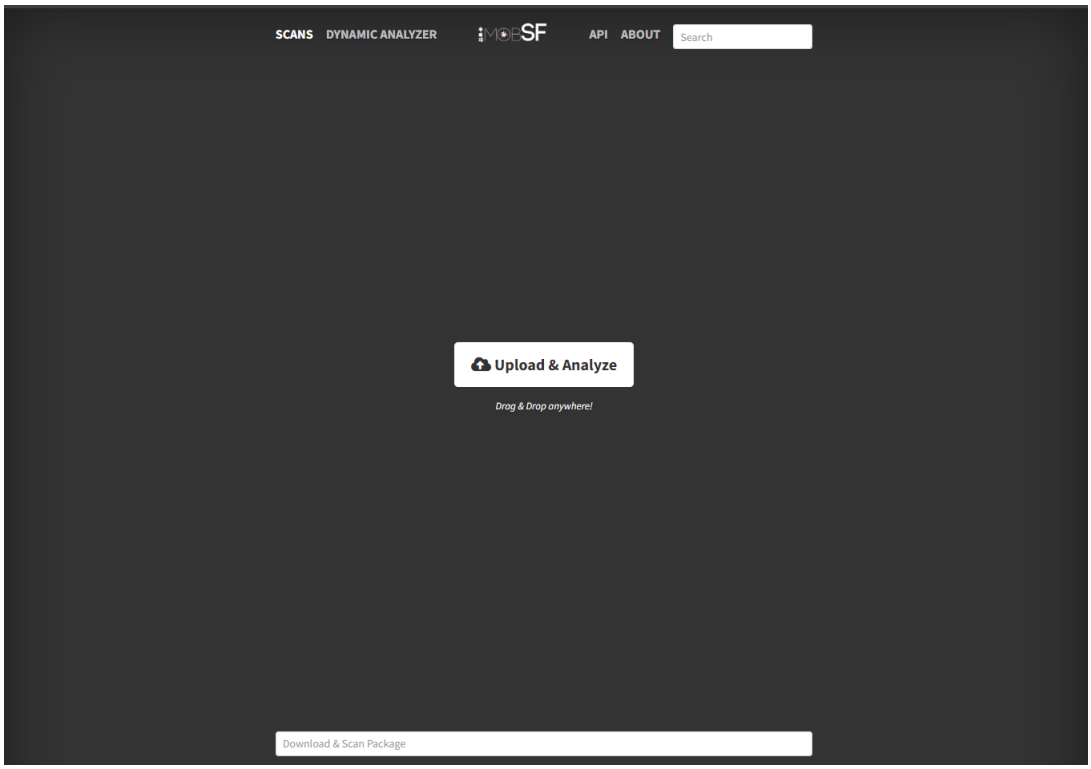
1. Navigate to: https://github.com/lucideus-repo/UnSAFE_Bank
    a. Click on ‘iOS’ -> ‘IPA File’ and click on UnSAFEBank.ipa



b. Click on the download button and it to the location of your choice.

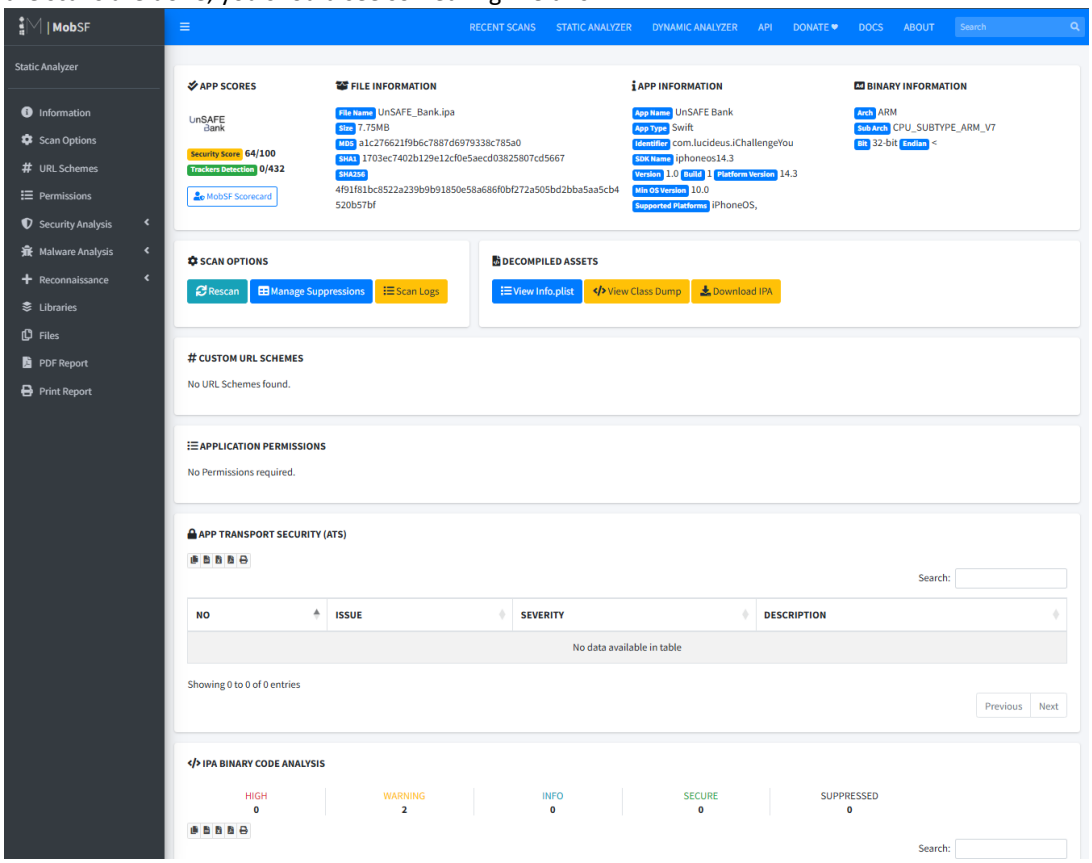


c. In a new tab navigate to: https://mobsf.live/



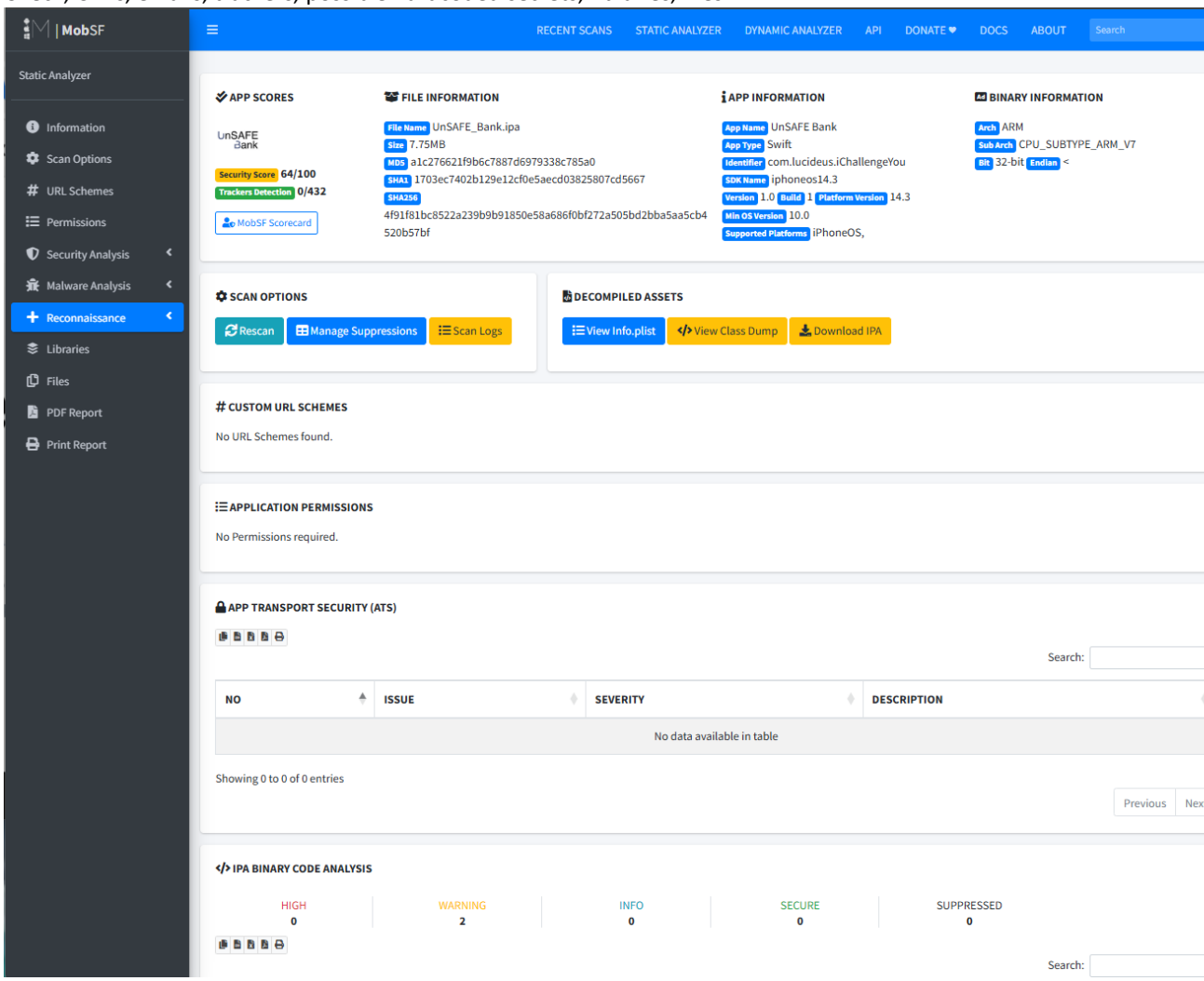
d. Drag and drop the UnSAFEBank.ipa on the 'Upload & Analyze' icon. This will start an auto scan of the iOS file. As Taylor might say, this could take a hot minute in case they have not seen this file before. But when

the scans are done, you should see something like this:



e. Take a moment to look at all the information in each of the sections. I am interested in the IPA Binary Code Analysis, IPA Binary Analysis, Dynamic Library & Framework Binary Analysis, Domain Malware

Check, URLs, emails, trackers, possible hardcoded secrets, libraries, files.



f. You can use the navigation bar to move between sections or download the entire report as a PDF.

2. Navigate to: https://github.com/t0thkr1s/allsafe/releases/latest/download/allsafe.apk

   a. This should automatically download the allsafe.apk to your default download locations.

   b. Repeat the scan from https://mobsf.live/ on this new file! (allsafe.apk)

   c. Please note this scan tool longer as the MobSF platform had many more tests for this file. Also I have noticed that the scan seam to run from bottom-up if you are playing along at home!

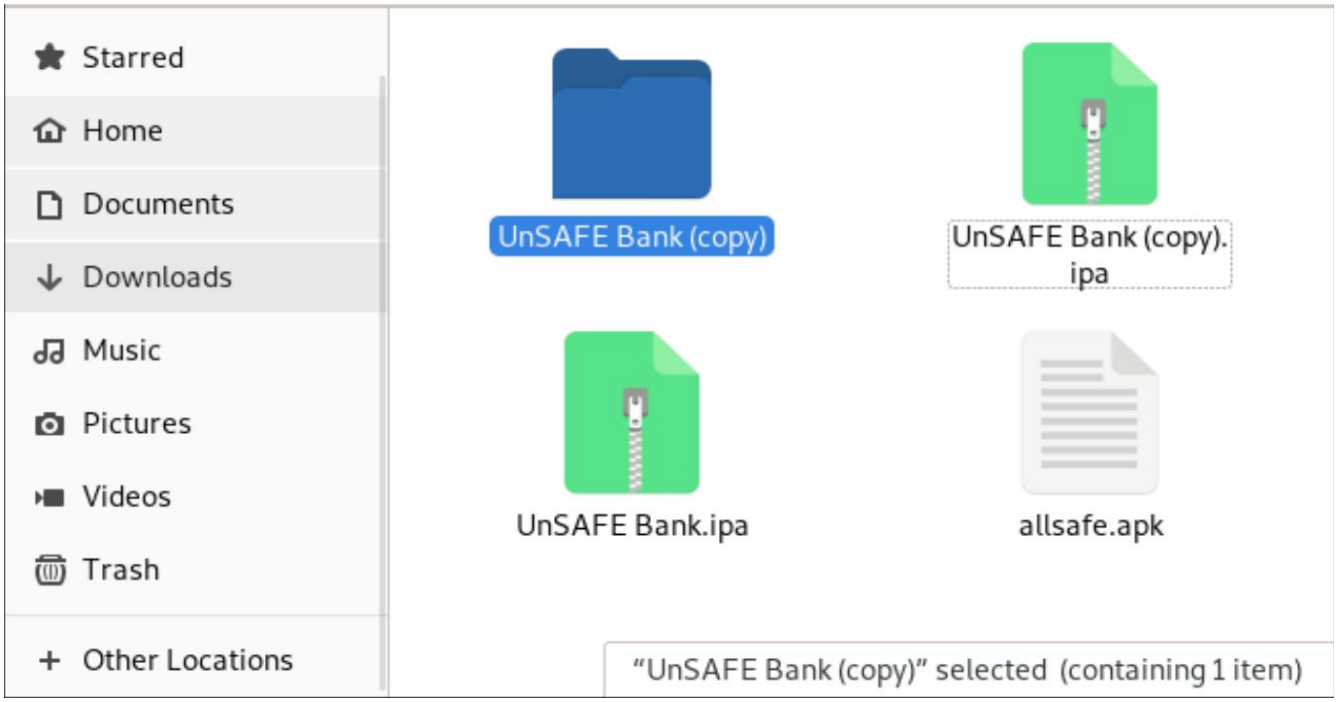
   d. Review the results, what interests you?

Challenge 3: Brief dive into an .ipa file

Explore a copy of UnSAFE Bank file.

1. In the file system explorer, make a copy of the 'UnSAFE Bank.ipa' file.

2. Double click on the copy of the 'UnSAFE Bank.ipa'. You will see a we now have a new folder



3. Doble click on the new folder -> 'Payload' -> 'UnSAFE Bank.app'. Here you will see several files that make up the app.

4. Double Click on the '_CodeSignature' folder. Then double click on the 'CodeResources' file. This will open it up in Firefox:


```
−<plist version="1.0">
  −<dict>
     <key>files</key>
    −<dict>
       <key>AppIcon20x20@2x.png</key>
       <data> Rv7YQhvY+yQfU9Vr/RtR0iSHEjQ= </data>
       <key>AppIcon20x20@3x.png</key>
       <data> yKbFuGiQq2kZHzFquPLN853lYfw= </data>
       <key>AppIcon29x29@2x.png</key>
       <data> frSANIdj1a3QXc0yZrYYiMigkTU= </data>
       <key>AppIcon29x29@3x.png</key>
       <data> CSMirpiFicmFS7jmRQjpqqA98Sw= </data>
       <key>AppIcon40x40@2x.png</key>
       <data> T/LhzJXPRnVfkD2GjTkMBNZu5E0= </data>
       <key>AppIcon40x40@3x.png</key>
```


5. Here we can see a list of what looks to be all of the files and a key that I am assuming is a hash. This artifact might be interesting to review to get a list of files in the app. More importaintly it shows that we can read the files in an unencrypted .ipa file.

6. Go back to where you made the copy of the UnSAFE Bank.ipa and delete the copied ipa file and the extracted folder we created in this section.

Challenge 4: CyberGym

APKtool

> “
> Apktool
> A tool for reverse engineering Android apk files
> - Disassemble
>   - Apktool can help disassemble resources to nearly original form.
> - Assemble
>   - Want to translate an app? Change a permission? Apktool can help you do that.
> - Analyze
>   - Not interested in rebuilding? Just attach -m to any disassemble command. Apktool will do its best to rip apart the resources and manifest for easy inspection.
>
> “

https://apktool.org/

A special thank you to the maintainers of lucideus-repo who maintains Cyber Gym.

> “
> CyberGym is a SAFE Security internal CTF for the SAFE Security hackers to learn and practice something new through Capture-The-Flag challenges. This repository consists of all the challenges part of the current CyberGym. Every challenge contains a setup guide and a challenge description so everybody can play and practice the challenges on their own. The challenges range from various categories of Web, Network, Mobile, Binary Exploitation, OSINT etc.
> “

https://github.com/lucideus-repo/cybergym/tree/master/CyberGym%202

We are going to use apktool to help solve ‘com.cybergym.lab1.apk’ mobile lab one challenge.

1. From your REMnux use firefox to go to: https://github.com/lucideus-repo/cybergym
2. Navigate ‘Mobile’ -> ‘lab1’ -> ‘app’ -> ‘release’ and download the ‘com.cybergym.lab1.apk’ file
3. Now we wat to bring up a terminal window.
    a. Type ‘pwd’ and hit enter to see your ‘present working directory’ / the directory that you are in
    b. Type ‘ls’ and hit enter to see the contents of your ‘present working directory’
    c. By default your document should have been downloaded to the ‘Downloads’ folder.
        i. Type ‘cd Downloads’ and hit enter

ii. Type 'ls' and hit enter and you should see our three files.

```
remnux@remnux:~$ pwd
/home/remnux
remnux@remnux:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
remnux@remnux:~$ cd Downloads
remnux@remnux:~/Downloads$ ls
 allsafe.apk   com.cybergym.lab1.apk  'UnSAFE Bank.ipa'
remnux@remnux:~/Downloads$
```

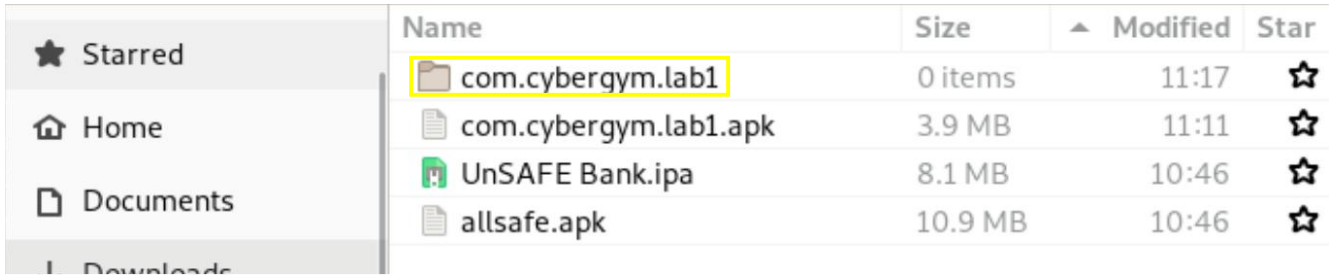
4. Now we want to run the command 'apktool d com.cybergym.lab1.apk' and hit enter. Remember most all Linux have tab to complete enabled which is your friend) This will decompile the file

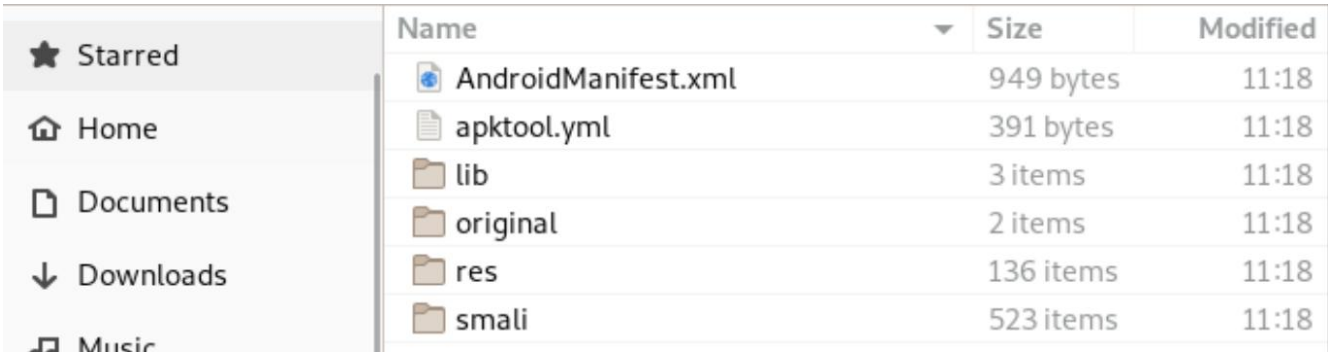
```
remnux@remnux:~/Downloads$ apktool d com.cybergym.lab1.apk
I: Using Apktool 2.9.3 on com.cybergym.lab1.apk
I: Loading resource table...
I: Decoding file-resources...
I: Loading resource table from file: /home/remnux/.local/share/apktool/framework/1.apk
I: Decoding values */* XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Regular manifest package...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
remnux@remnux:~/Downloads$
```

5. You can use the 'ls' command and see we now have a folder (in blue in this distro) named 'com.cybergym.lab1.apk'

```
remnux@remnux:~/Downloads$ ls
 allsafe.apk   com.cybergym.lab1   com.cybergym.lab1.apk  'UnSAFE Bank.ipa'
remnux@remnux:~/Downloads$
```

6. Now we can go to our file explorer and validate and enter this new folder:





7. Success!!!

Jadx

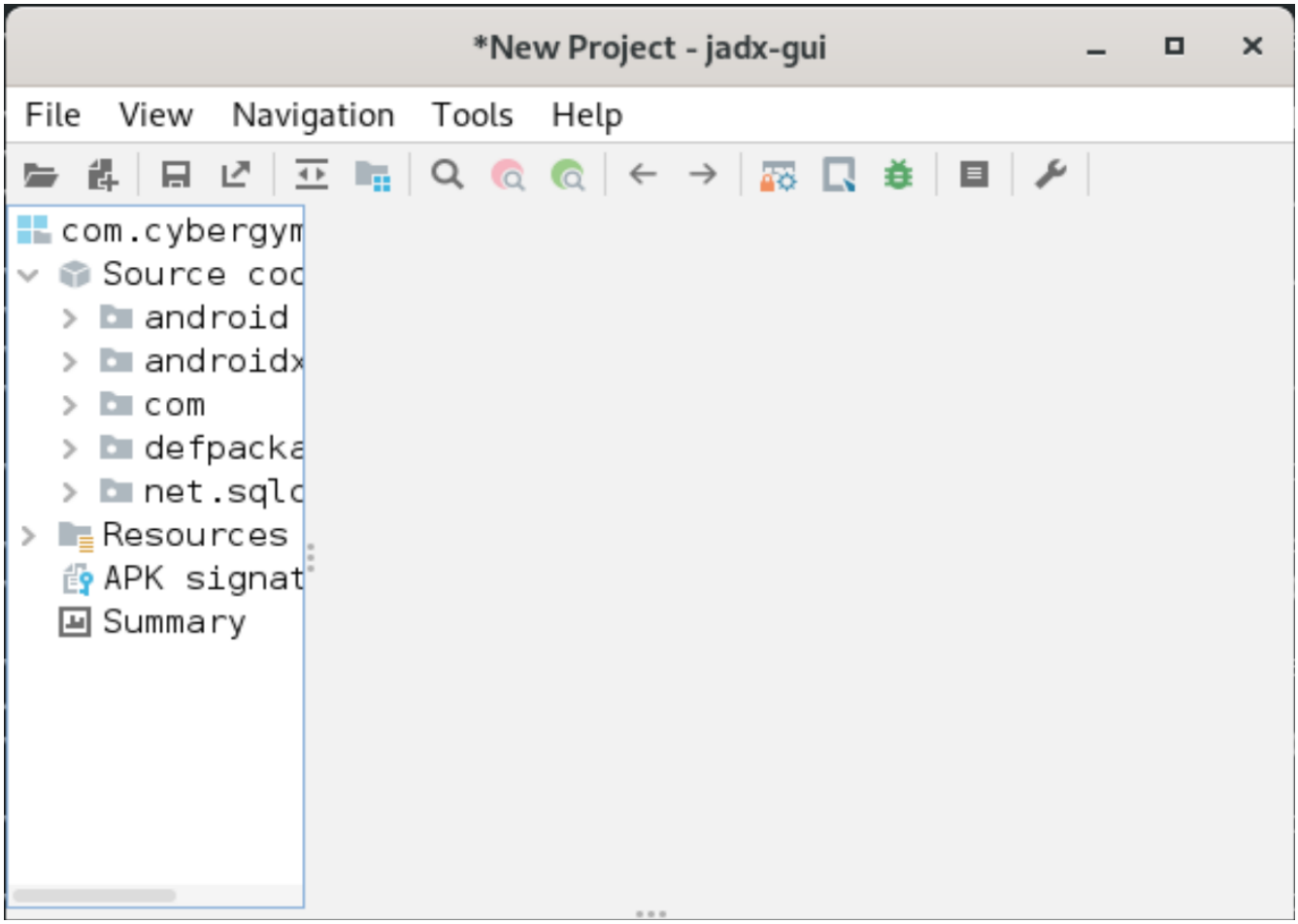
Jadx is a popular, open-source **Dex to Java decompiler** tool used to produce readable Java source code from Android Dalvik Executable (DEX) files, as well as APK, JAR, and AAR files. It includes both a command-line interface and a graphical user interface (GUI). Created by Android Dex.

To make thing much easier lets use the GUI version.

1. Type 'jadx-gui com.cybergym.lab1.apk' and hit enter.

```
remnux@remnux:~/Downloads$ jadx-gui com.cybergym.lab1.apk
```

2. A new pop up should appear.



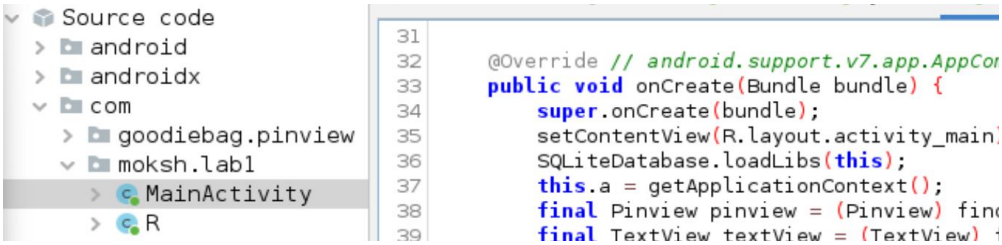
3. Explore the file tree and click on some files. You can see we have human readable code!
    a. Object icon cheat sheet
        i. 'C' icons indicates "C" icon indicates a synthetic, non-printable, or otherwise incorrectly named class/method that JADX has renamed
        ii. 'I' icon indicates what jadx believes to be an icon within a class (not always correct.)
        iii. '@' icon indicates an annotation within a class
        iv. 'f' icon indicates a field or varable within a class
        v. 'm' icon indicates a methode in the class

vi. ‘m’ icon indicates an error during decomplilation

b. When jadx makes changes to a file it will put comments in shown in green with format ‘/*text*/

```
/* renamed from: ay$a */
/* loaded from: classes.dex */
public static final class a {
    public static final int alpha = 2130837543;
    public static final int coordinatorLayoutStyle = 213083
    public static final int font = 2130837627;
```

4. For this lab we need to migrate ‘Source Code’ -> ‘com’ -> ‘mpksh.lab1’



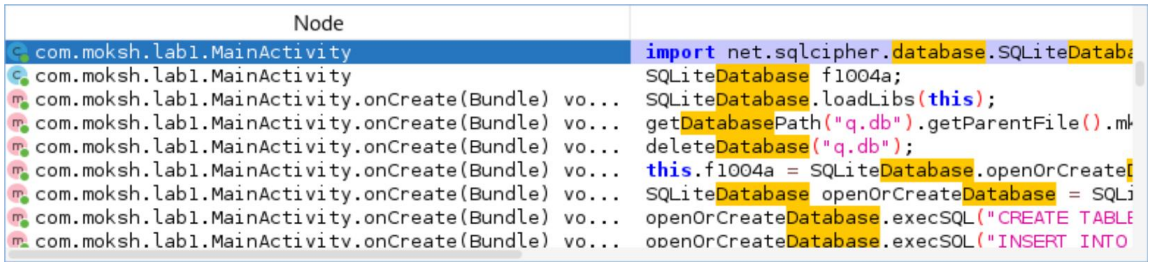
5. In looking at the imports we can see what classes or static members we are using.

   a. Line 12: has something called ‘com.goodiebad.pinview.Pinview’. a simple google on ‘Pinview’ shows that it is a common UI component for apps entering PINS/passcode.

   b. Line 14: has something called ‘net.sqlcipher.database.SQLiteDatabase’ so it looks like we might have connections to a SQLite Database

6. On the toolbar we can click the magnifying glass. A search tool will show up for the entire apt file will open. Leave it as is, but in the search for text type ‘database’ and click search. You should see over 650 reports. You wll see some are just comments (in green text), but others appear to be actual configuration.

   a. We are real interested in the where the node is ‘com.moksh.lab1.MainActivity*’ as this file is the scope of the lab. Now we see are down to 12 entries.



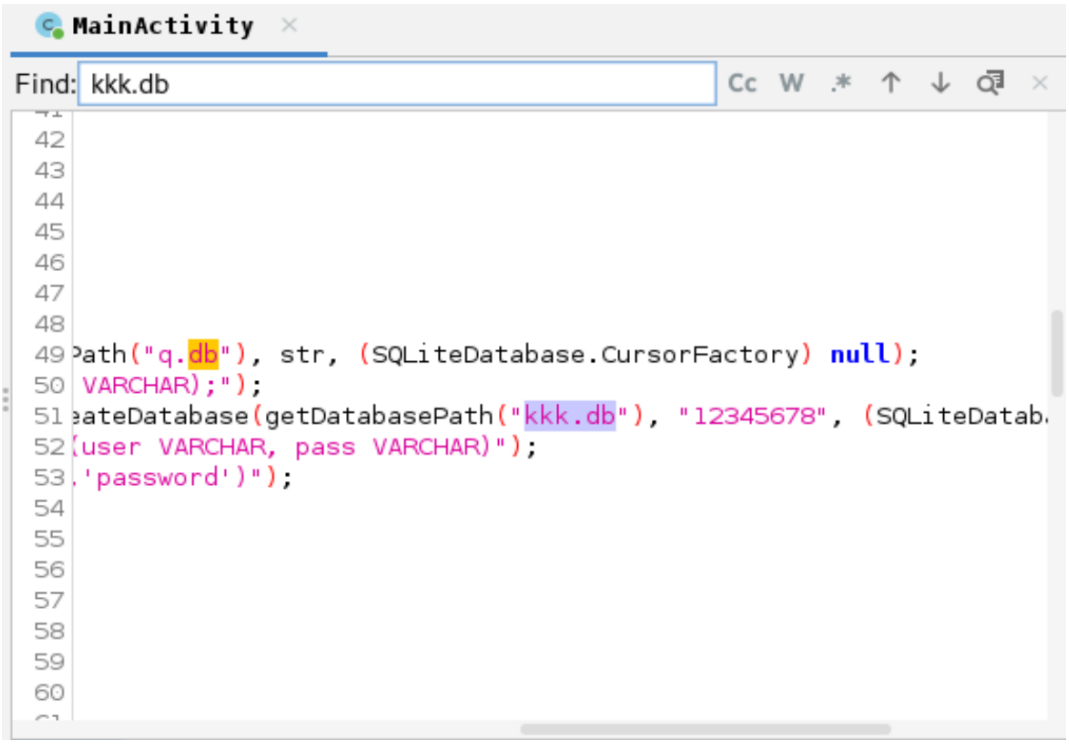
   b. Three entries stand out to me:

```
SQLiteDatabase.openOrCreateDatabase(getDatabasePath("kkk.db"), "12345678",
("CREATE TABLE IF NOT EXISTS name(user VARCHAR, pass VARCHAR)");
("INSERT INTO name VALUES('moksh','password')");
```

   c. Close this window

   d. Tap ‘Control + F’ and now you will see a find field has opened up. In one of the screencaps you will see we have a command that indicates that we are ‘openOrCreateDatabase’ with a filename ‘kkk.db’ and a second parameter ‘12345678’ which might look to be a password or other authenticator. In the

MainActivity find, I am going to search for ‘kkk.db)



e. Looks like we found it at line 51! Intesting enough the other two lines I was interested in are immediately below the command we were looking at in line 52 and 53. Line 54 looks like we are closing the connection

f. As I look at the start of the ‘try’ function it looks we are also doing something with ‘q.db’ in line 47 and 48. Interesting.

g. We can see the SQLiteDatabse f1004a is defined on line 24. This ;f1004a’ can be found in line 49, 50, 60, 65, 103, and 125.

```
24      SQLiteDatabase f1004a;
25      int b;
26      int c = 2;
```

h. So in conclusion line 51 attempts to open a database connection. Line 52 looks to create a table named ‘name’ with two columns ‘user’ and ‘pass’. Line 53 looks to add ‘moksh’ and ‘password’ into that table.

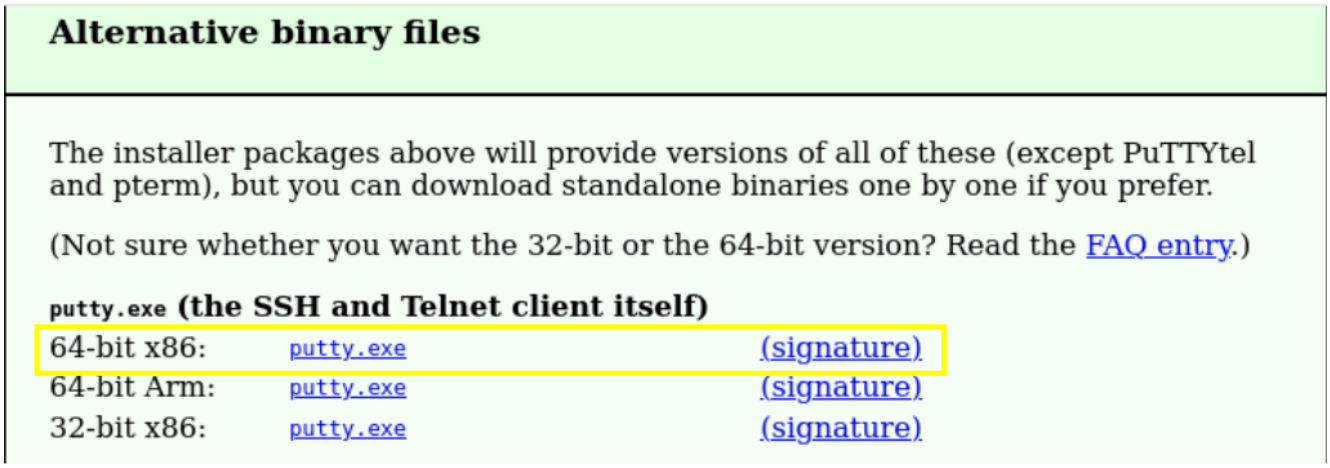
Challenge 5: Ghidra

This challenge is greatly inspired by:

https://blogs.blackberry.com/en/2019/07/an-introduction-to-code-analysis-with-ghidra

For this lab we are going to use a version of putty portable. Putty is an ‘old school’ Program that allows you to make SSH, Serial, Telnet, Rlogin, SUPDUP, Raw, and bare ssh-connections. I was introduced to when I started working in the networking / security world. I toyed around with leveraging a real malicious file but I did not want the conference (or more importantly me) to accept the responsibility. I choose putty because it is free and relatively small.

from your REMnux download the portable version of putty 64-bit x86 off this sites.



Alternative binary files

The installer packages above will provide versions of all of these (except PuTTYtel and pterm), but you can download standalone binaries one by one if you prefer.

(Not sure whether you want the 32-bit or the 64-bit version? Read the FAQ entry.)

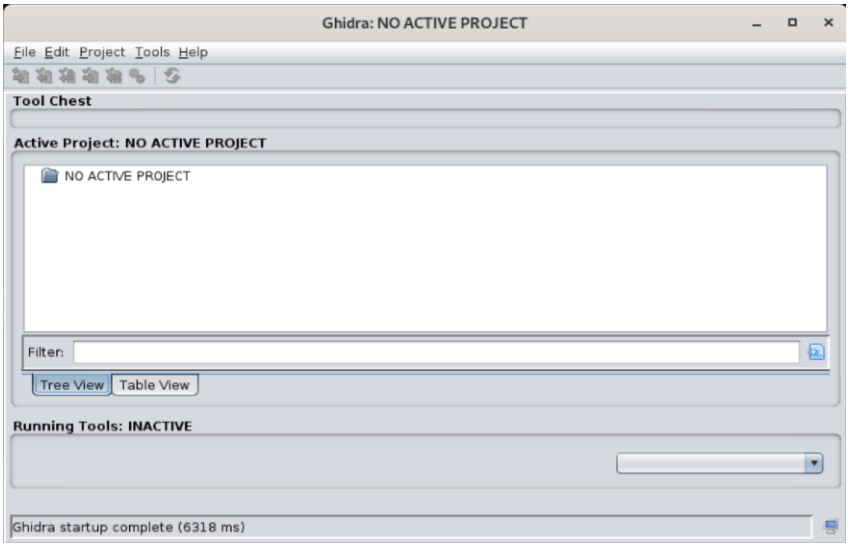
putty.exe (the SSH and Telnet client itself)

64-bit x86:	putty.exe	(signature)
64-bit Arm:	putty.exe	(signature)
32-bit x86:	putty.exe	(signature)

https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html

In your REMnux machine click 'Activities' -> 'Show Applications' (the icon with 9 dots in a square' -> 'Ghidra' to launch Ghidra.

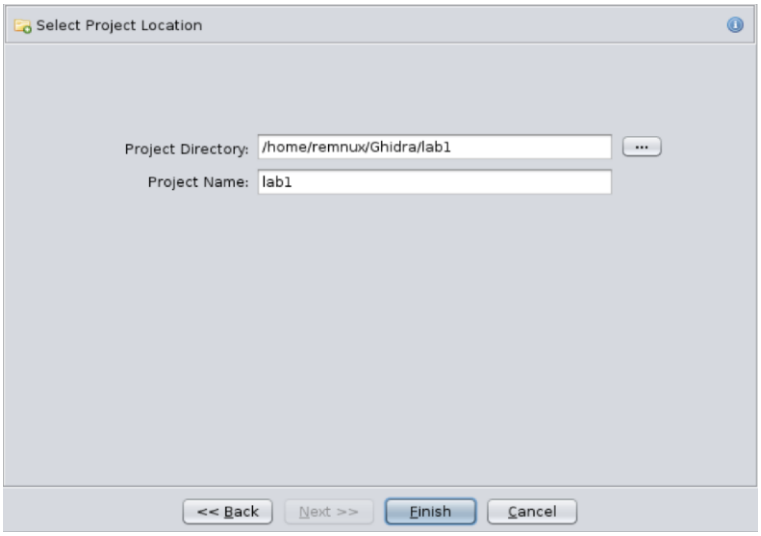
Close out of the 'Tip of the Day' and the manual if they appear. You should see that you have no active projects.



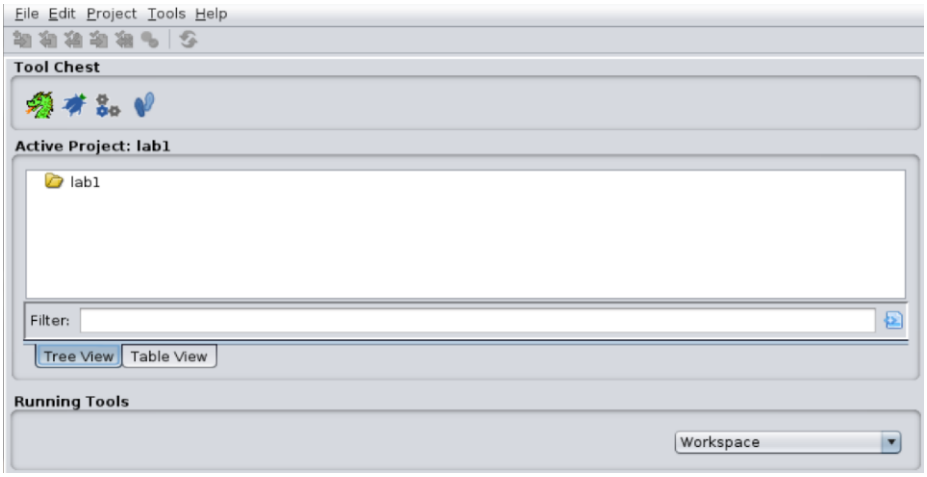
Click 'File' -> 'New Project' to lauch the new Project pop up.

1. Select 'Non-Shared Project' on the first window and press 'Next'
2. Pres the three dot button on the right of Project Directory.
    a. This will launch a file browser.
    b. Click on 'Home'.
    c. In the upper right hand of this window you will see a folder with a + on it. Click on it to create a new folder here named 'Ghidra'
    d. Double Click on the 'Ghidra' to enter this folder. Here create a new folder named 'lab1'
    e. Single click on 'lab1' and then click ;Select Project Directory' button on the bottom of this window.

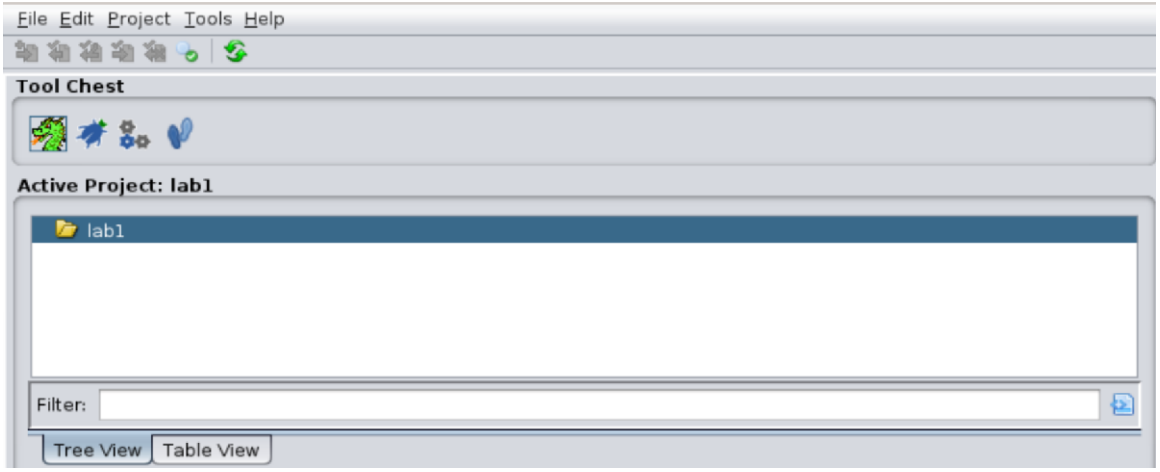
3. Back on the ‘Select Project Location’ window, tyle ‘lab1’ in ‘Project Name’. Click finish



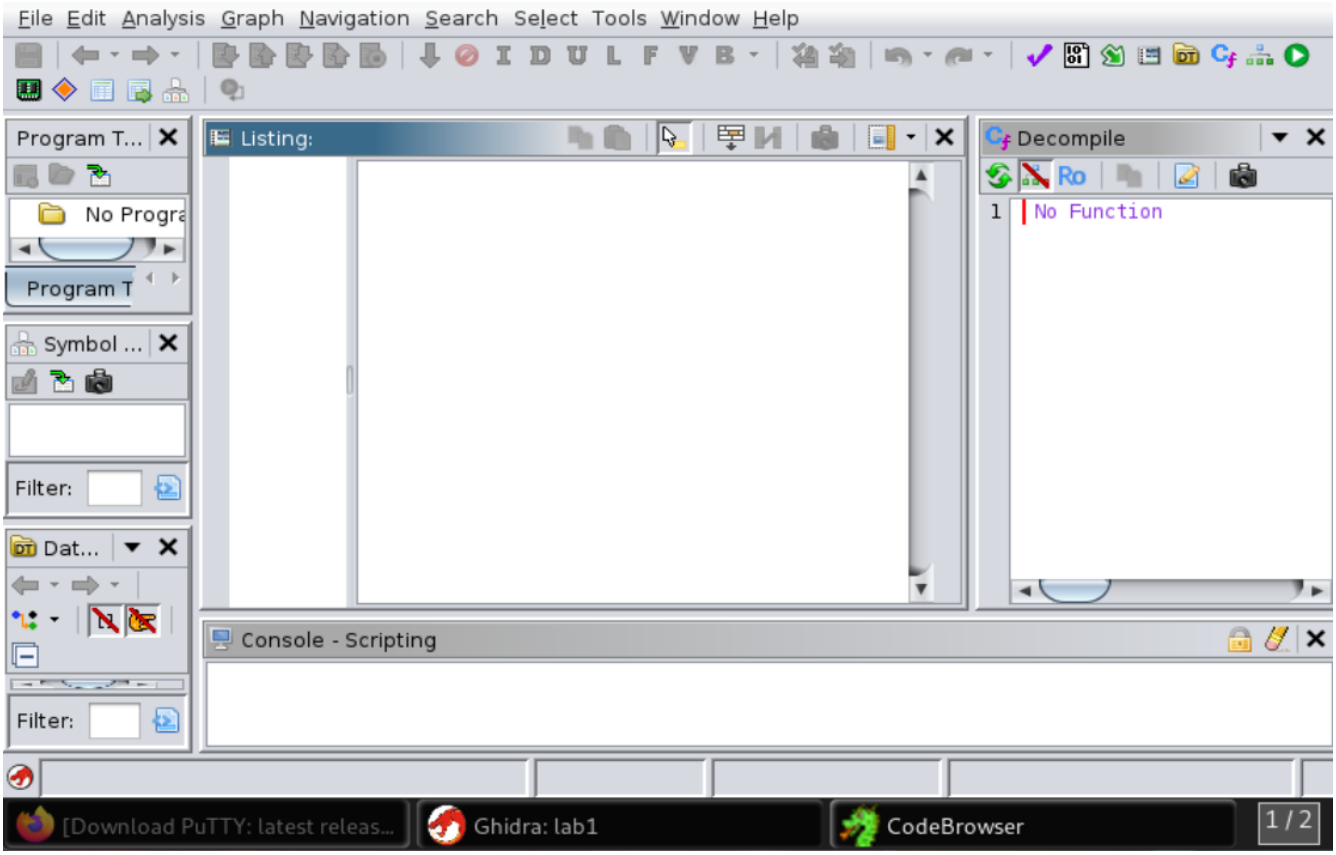
4. Now you should be at the main Ghidra welcome screen.



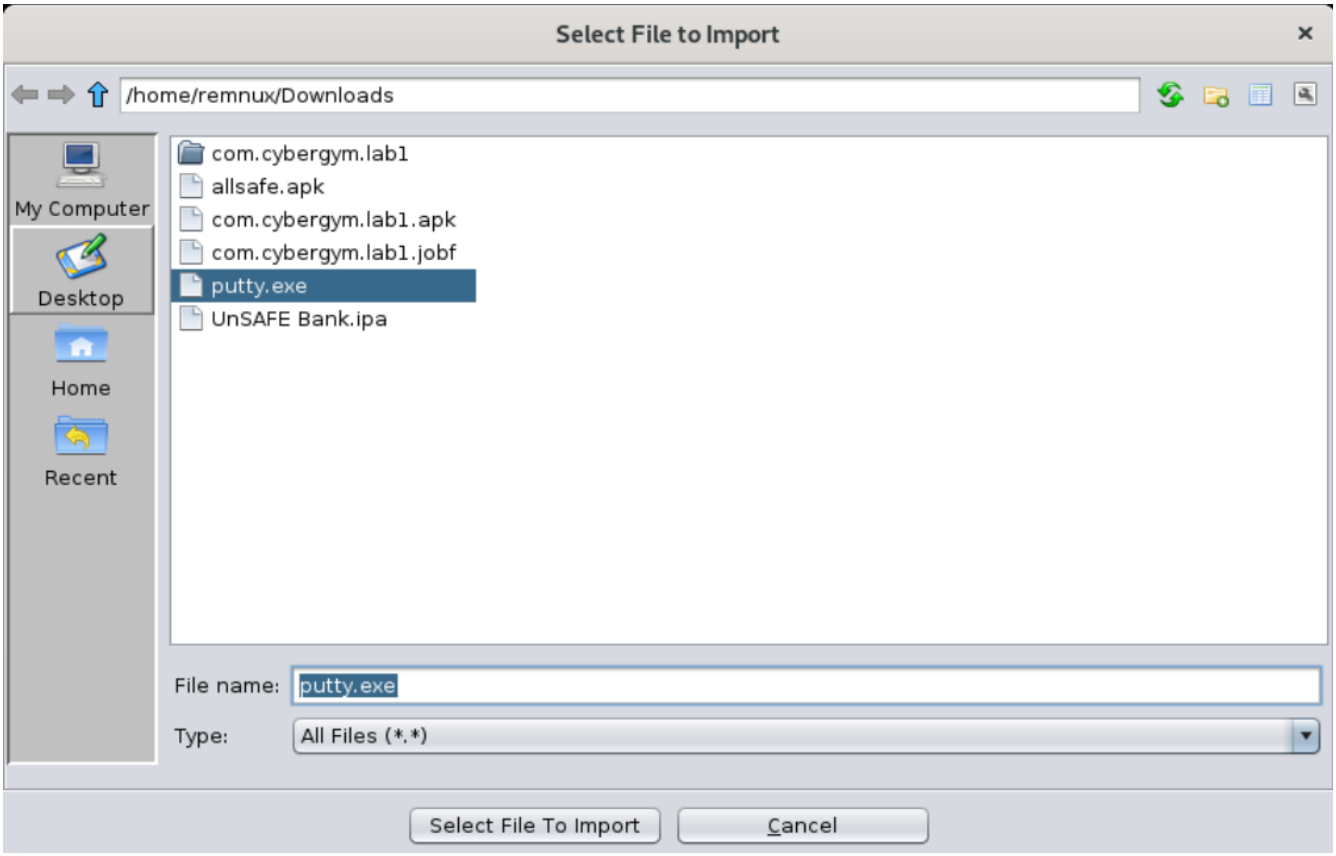
5. Click on ‘lab1 ‘ and then the ‘code browser’ button (icon on the toolchest that looks like a dragon)



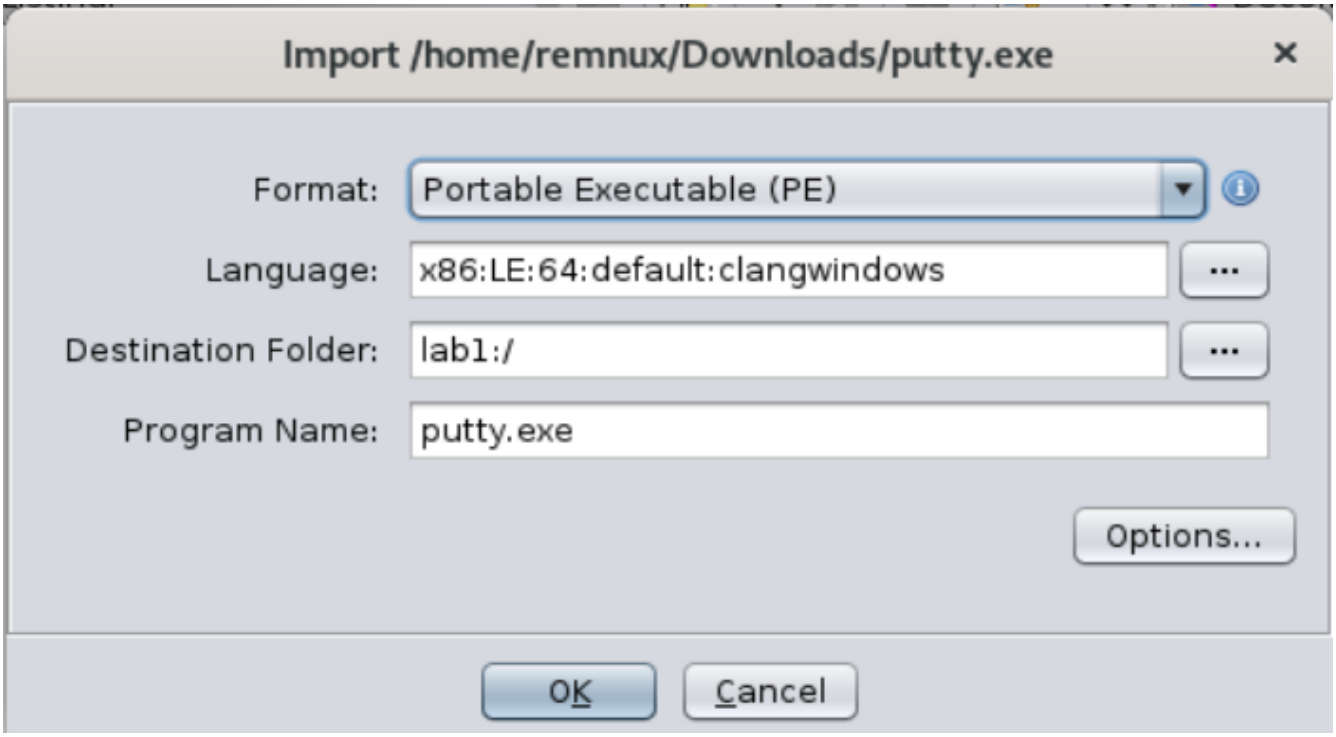
Sometimes the code browser lauches being the initial Ghidra. Look at the bottom open apps bar and click on ‘CodeBrowser’. It should look like this.



1. Click ‘File’ -> ‘Import File’

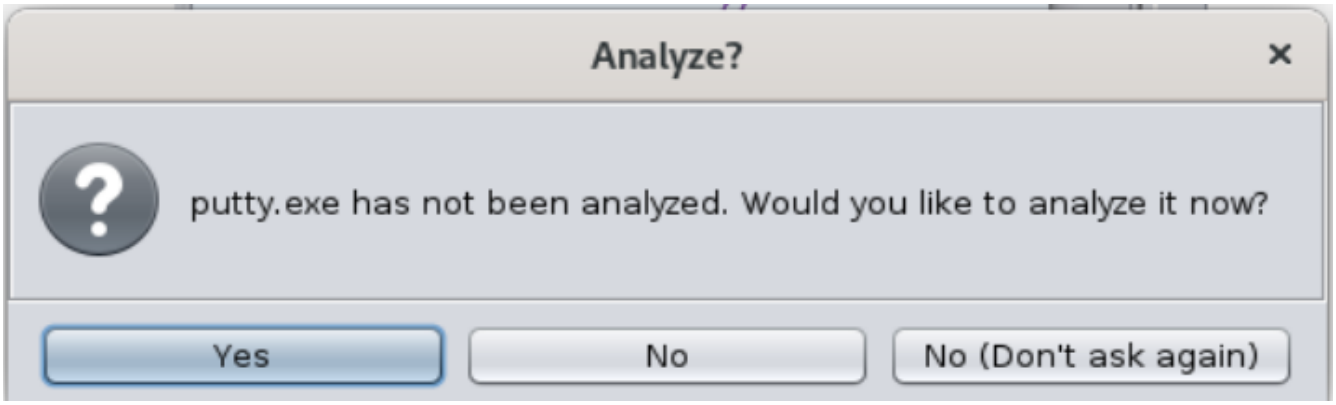


2. Click on your ‘putty.exe’ and then click on ‘Select File to Import’
3. Ghidra has already filed this out for us, but if it is wrong you can customize. (In this case it is correct) Click ‘Ok’

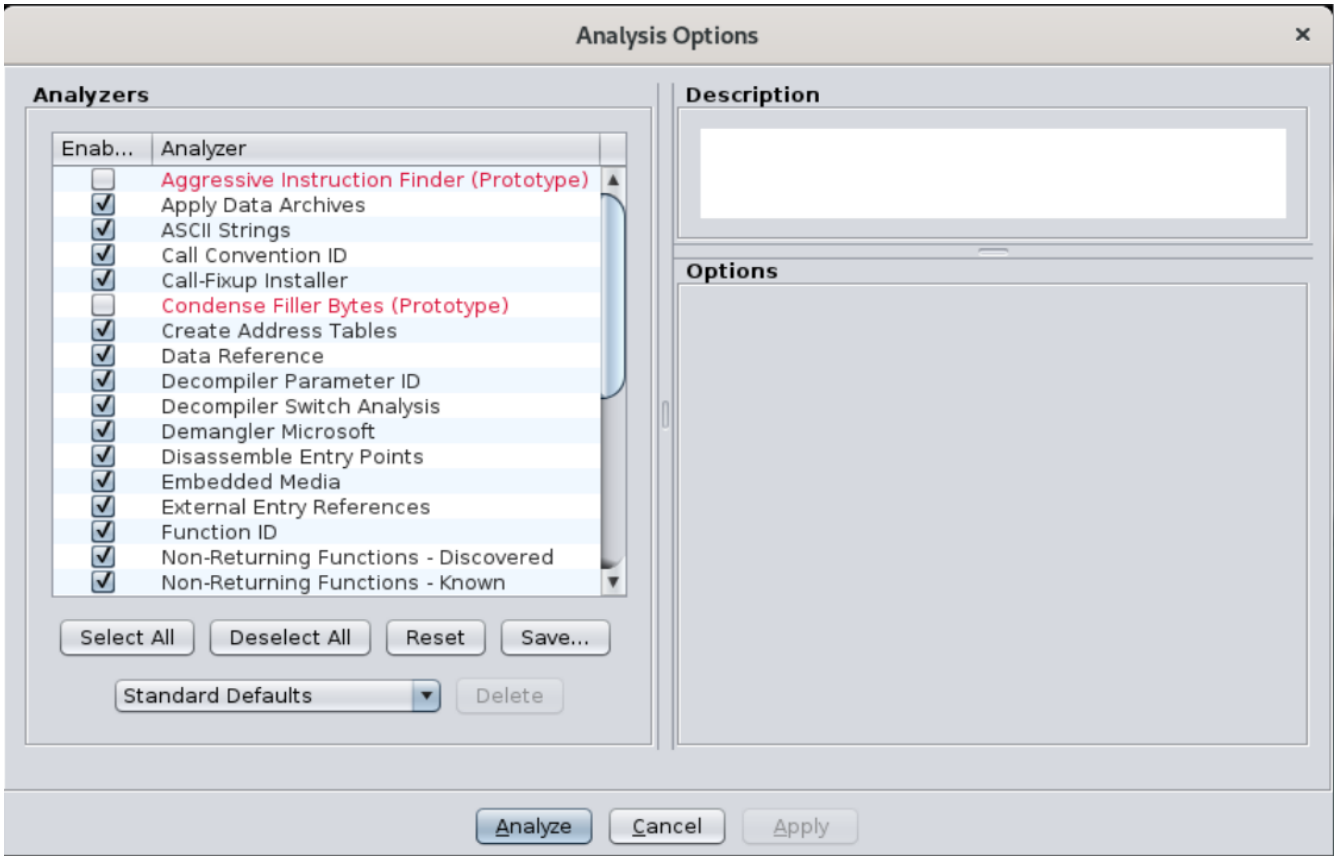


4. The file will start importing

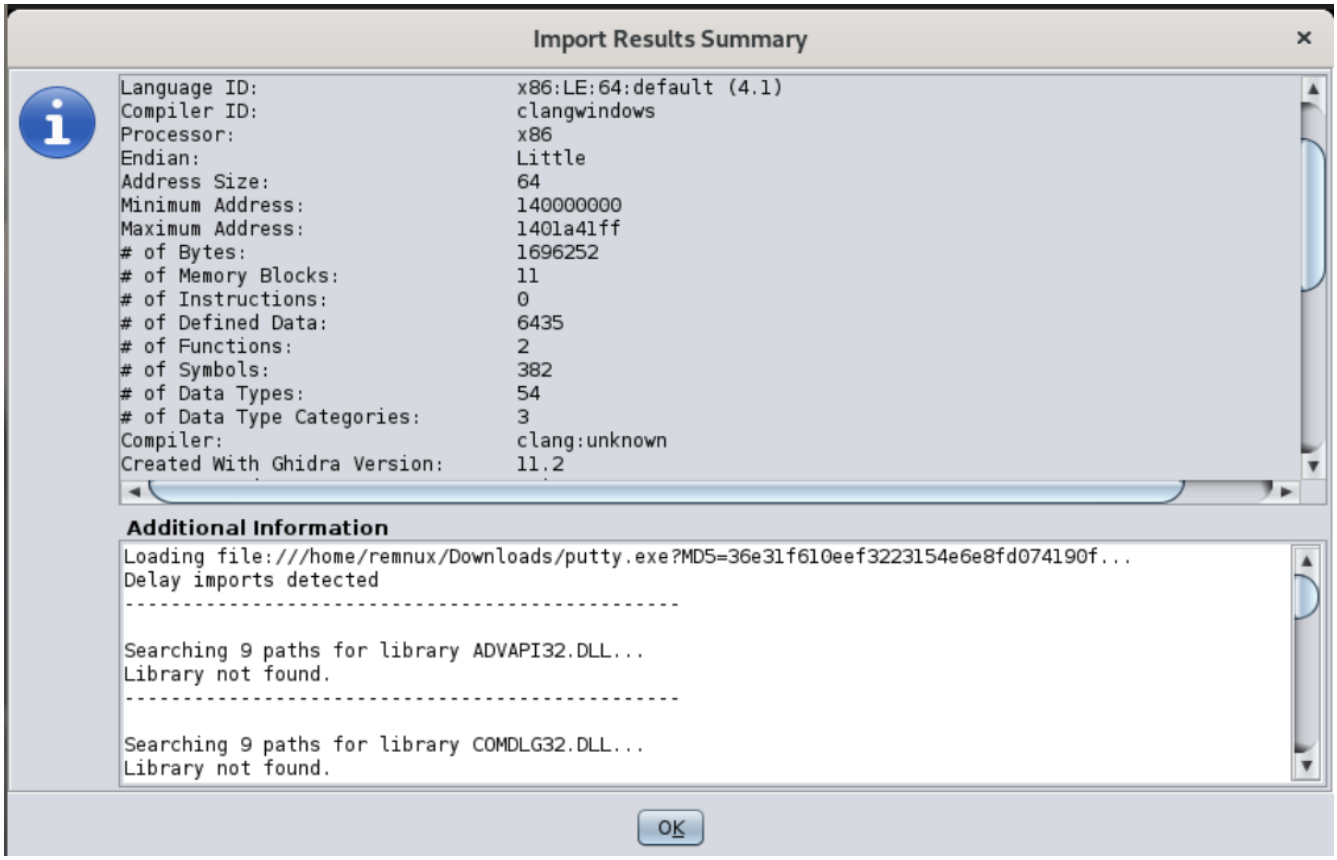
5. Click ‘Yest’ to allow Ghidra to analyze the file.



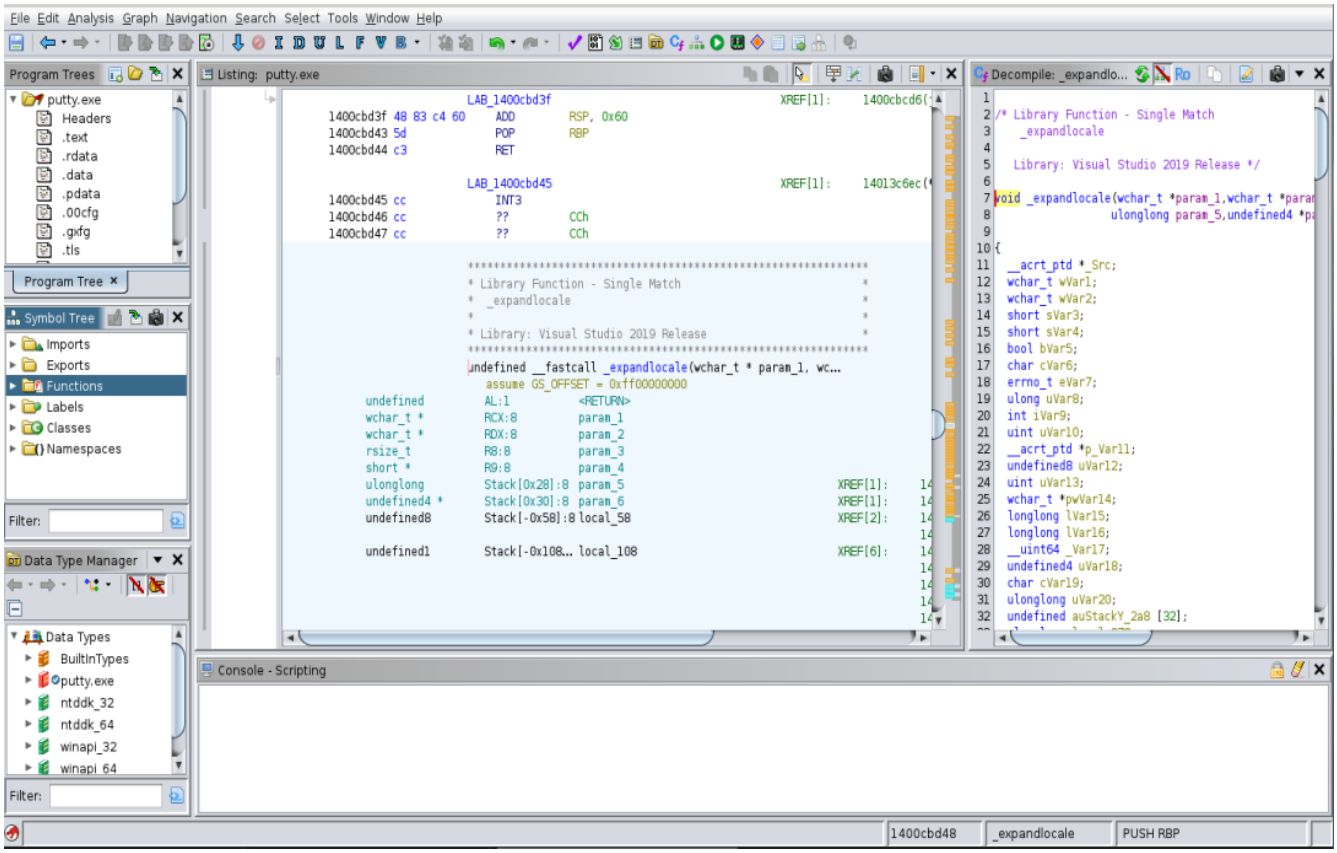
6. We are going to trust the default Analyzers. (In the real workd you can customize this). Click Analyze’ at the bottom of this screen.



7. Next we have the summary screen. Take a few moments to read the summary. Then click ‘Ok’ to close this view.



8. It is at this time you might want to go to ‘Activities’ -> ‘Show Applications’ -> ‘Settings’ -> Display’ and change your resolution to you can see more. This is what you should see.



9. Now this is where I get a bit out of my comfort zone with assembly language in the ‘Listing’ window. Ghidra attempts its best to translate the code int a more human readable version in the Decompiler window.
    a. Assembly language is a low-level programming language that provides a human-readable representation of a computer's machine code instructions. Unlike high-level languages like Python or C++, assembly directly corresponds to specific hardware operations, allowing for maximum performance, precise memory control, and efficient hardware manipulation.
    b. Key Concepts:
        i. Mnemonics: Short abbreviations (e.g., MOV for move, ADD for addition) used to represent binary machine code.
        ii. Registers: Fast, internal processor storage locations used for intermediate calculations.
        iii. Assembler: A tool (e.g., NASM, MASM, GAS) that translates assembly code into binary machine code.
        iv. Architecture Specific: Assembly code written for one processor (e.g., x86) will not work on another (e.g., ARM).
10. In the program tree you can click on any of the detected items and it will jump to that location. Try that on any of the detected items.
11. The same is try in the symbols tree. Try to double click some of the detected items.

12. Now that you have the basics you need to use Ghidra. Many Reverse Engineers, mush more talented than me, much more, or at least that is what people tell me have written detailed Ghidra reverse engineering write ups for free on the internet.

More resources:

- https://github.com/rednaga/training/tree/master/DEFCON23
- https://github.com/onethawt/reverseengineering-reading-list
- Modern X86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX by Daniel Kusswurm
- Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software by Michael Sikorski & Andrew Honig
- Reversing: Secrets of Reverse Engineering by Eldad Eilam

Until Next time!